

# Docker Build

---

[github.com/O-clock-Cyber/slides/blob/master/slides/13.SC4\\_Conteneurs-et-orchestration/02.-Docker-build-et-Compose/slides.md](https://github.com/O-clock-Cyber/slides/blob/master/slides/13.SC4_Conteneurs-et-orchestration/02.-Docker-build-et-Compose/slides.md)



---

## Docker : Build & Compose

*Construire ses images et orchestrer ses conteneurs*

### Rappel express

---

--

### Ce qu'on sait déjà

---

- **Image** = la recette (immuable) 
- **Conteneur** = le plat servi (instance d'une image) 
- **DockerHub** = le supermarché des images 

Aujourd'hui, on apprend à **cuisiner nos propres recettes** ! 

### Le Dockerfile

---

*La recette de votre image*

--

## C'est quoi ?

---

Un fichier texte qui décrit **étape par étape** comment construire une image Docker.

Comme une recette de cuisine : on part d'une base, on ajoute des ingrédients, on configure 🧑🍳

--

## Les instructions essentielles

---

- **FROM** — l'image de base (point de départ)
- **RUN** — exécuter une commande pendant le build
- **COPY** — copier des fichiers locaux dans l'image
- **WORKDIR** — définir le répertoire de travail
- **EXPOSE** — déclarer un port
- **CMD** — la commande lancée au démarrage du conteneur

--

## Exemple : un serveur web simple

---

```
FROM nginx:alpine
```

```
COPY ./mon-site/ /usr/share/nginx/html/
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

*4 lignes et on a une image prête à servir notre site ! 🚀*

--

## Exemple : une app Node.js

---

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["node", "server.js"]
```

*On copie d'abord le package.json pour profiter du **cache Docker** sur les dépendances 🧠*

## Transformer un Dockerfile en image

--

## La commande magique

---

```
docker build -t mon-app:v1 .
```

- **-t mon-app:v1** → le nom et le tag de l'image
- **.** → le contexte de build (le dossier courant)

--

## Les layers (couches) 🍷

---

Chaque instruction du Dockerfile crée une **couche** :

- Les couches sont **mises en cache** 💾
- Si rien ne change → pas de rebuild
- D'où l'importance de l'**ordre des instructions**

*Astuce : mettez les éléments qui changent le moins souvent en premier !*

--

## Le .dockerignore 🚫

---

Comme un **.gitignore**, mais pour Docker :

```
node_modules
.git
.env
*.md
```

Évite d'envoyer des fichiers inutiles dans le contexte de build.

## Bonnes pratiques Dockerfile 📋

---

--

## Les réflexes à avoir

---

- 🍷 Utiliser des images **légères** (**alpine**)
- 📦 **Minimiser** le nombre de couches (combinaison des RUN)
- 🔒 Ne **jamais** mettre de secrets dans l'image
- 👤 Utiliser un **utilisateur non-root** quand c'est possible

--

## Exemple : combiner les RUN

---

❌ Mauvais :

```
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y vim
```

✅ Mieux :

```
RUN apt-get update && apt-get install -y \
    curl \
    vim \
    && rm -rf /var/lib/apt/lists/*
```

## Docker Compose

---

*Orchestrer plusieurs conteneurs*

--

### Le problème

---

Une application web moderne, c'est souvent :

-  Un frontend (Nginx, Apache...)
-  Un backend (Node, PHP, Python...)
-  Une base de données (MySQL, PostgreSQL...)
-  Un cache (Redis...)

Gérer tout ça à la main avec `docker run` ? 😞

--

### La solution : Docker Compose

---

Un fichier `docker-compose.yml` qui décrit **toute l'architecture** :

Quels conteneurs, quels réseaux, quels volumes, quelles dépendances.

*Analogie : le chef d'orchestre qui coordonne tous les musiciens* 🎵

--

## Syntaxe YAML — rappel express

---

```
# Un commentaire
cle: valeur
liste:
  - element1
  - element2
objet:
  sous_cle: sous_valeur
```

*C'est comme du JSON, mais plus lisible (et sensible à l'indentation !)*

## Exemple concret

---

*Une stack LAMP en Docker Compose*

--

## Le fichier docker-compose.yml

---

```
services:
  web:
    image: php:8.2-apache
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
    depends_on:
      - db

  db:
    image: mariadb:11
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: myapp
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:

--
```

## Décryptage

---

- **services** : les conteneurs à lancer
- **image** : quelle image utiliser
- **ports** : mapping host → conteneur
- **volumes** : persistance des données
- **depends\_on** : ordre de démarrage
- **environment** : variables d'environnement

## Les commandes Compose

---

--

### Les essentielles

---

- `docker compose up -d` → tout lancer en arrière-plan 
- `docker compose down` → tout arrêter et nettoyer 
- `docker compose ps` → voir l'état des services 
- `docker compose logs -f` → suivre les logs en temps réel 

--

### Les pratiques

---

- `docker compose build` → rebuild les images custom 
- `docker compose exec web bash` → entrer dans un conteneur 
- `docker compose restart web` → redémarrer un service 
- `docker compose pull` → mettre à jour les images 

## Réseaux dans Compose

---

--

### Par défaut

---

Docker Compose crée automatiquement un réseau pour chaque projet.

Les conteneurs se trouvent par leur **nom de service** !

```
# Dans le code PHP :  
# host = "db" (pas "localhost" ni "127.0.0.1")  
$pdo = new PDO("mysql:host=db;dbname=myapp", "root", "secret");
```

--

### Réseaux custom

---

```
services:  
  web:  
    networks:  
      - frontend  
      - backend  
  db:  
    networks:  
      - backend  
  
networks:  
  frontend:  
  backend:
```

Le service *db* n'est pas accessible depuis le réseau *frontend* → segmentation ! 🔒

## Volumes & persistance 💾

---

--

### Le problème

---

Un conteneur est **éphémère** : si on le supprime, les données disparaissent 🧠

Solution : les **volumes** !

--

### Deux types

---

- **Named volumes** : gérés par Docker, persistants *db\_data:/var/lib/mysql*
- **Bind mounts** : lien vers un dossier de l'hôte *./src:/var/www/html*

*Bind mounts* = pratique en dev. *Named volumes* = recommandé en prod

## Variables d'environnement 🔒

---

--

### Le fichier .env

---

```
MYSQL_ROOT_PASSWORD=supersecret
MYSQL_DATABASE=myapp
APP_PORT=8080
```

```
services:
  db:
    image: mariadb:11
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
  web:
    ports:
      - "${APP_PORT}:80"
```

⚠ Ne committez **jamais** le fichier *.env* !

## En résumé 📝

---

- 📄 **Dockerfile** = recette pour construire une image
- 🔨 **docker build** = cuire la recette
- 🎵 **Docker Compose** = orchestrer plusieurs conteneurs
- 📋 **docker-compose.yml** = la partition de l'orchestre

Prochaine étape : l'orchestration à grande échelle avec Docker Swarm ! 🚀